SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

CSA3714 - SOFTWARE TESTING

ASSESSMENT TOOLS CO1 - AT1

Name: Darani E

RegNo: 192312120

7) 4, 8, 12, 14, 17

A Difference Between Software Testing & Debugging:

In Aspect of Software Engineering Testing & Debugging seems to be similar but it varies Software Testing comes Under SDLC [Software Development life cycle] Software Testing is a Process where finding of defects taken place, executing or evaluating Software to determine whether it meets Specified requirements, to defect detection.

for an Example Testess enter valid Login Data such as Username & password but the Software display the error message. The tester will have the record of failures or bug.

-> Debugging is process of Correcting, Analyzing the Cause of a defect in the Software.

for an Example if a testes finds bug, developers work on login code & discover Username function with incorrect condition. Developers Corrects the code.

-> In terms of Software Quality:

Testing helps identify problem before Software Get Released, Debugging helps developers (Understand) the cause of the problems & correct the bugs which get identified by Testers. Both Activities Software Testing & Debugging is essential Because detecting a defect does'nt Automatically fix it.

→ Effects of Confusing Software Testing & Debugging:

If a person who confuses & treats Debugging is same as testing the some defects may remain undetected, Developers may fix defects without finding a root cause, which results repetation of Same defects, Regression testing may be neglected and Software Quality decreased.

Testing detects failures, Debugging identifies & Correct their Causes. Integrating Both through a Structured workflow Improves defect resolution, reduce regression, enhance Software Quality.

---

8) e-commerce platform inconsistent behaviours Across Browsers:

Cross Browsers is ~~a Black~~ Verifies that a web Application work consistently without any defects on a different Browsers, different Browser versions, different Operating System, different Screen sizes, for an example a web browser to manage blood Donor is been launch without Cross Browser testing Report defect, The Defects may occur which affect client.

How Defects occur:

* The Cross Browser defects originates from Different Browser versions. Suppose web browser for Blood Donor was supported firefox version 12.2, may not support for firefox 12.1.2.

* Different Operating Systems, if application is built to support all operating System. but at testing possibility of occuring of

defects

* Unsupported Implementation HTML, JavaScript.
* Different font & layout behaviour
* Browsers Specific API Behaviour
* Response design issue.
* Difference in Security levels of Browsers & privacy Settings.

Testing Principles to design a Strategy:

Context Dependent Testing:

Precedence of Context Dependent Test, In a ATM Machine Application, Give Special Attention to the payment security & Checkout reliability.

Early testing:

Check Browsers Compatibility Before launch of the Software, Early testing is easy affordable rather than testing after the Software Release.

Defect Clustering:

Grouping of defects focus on components where compatibility defects repeatedly occurs such as checkout & payment pages.

Pesticide Paradox:

Regularly Update the test cases as Browsers feature Change.

Cross-Browser Testing Strategy:

1: Identify Target Browsers: Select supported Browsers, along with desktop & mobile platforms.

2: Prepare the test Enviornment: Use Actual device or Browser for setting an environment with Required browser versions.

3: Design Test Cases: Which Browsers Versions, font size & Layout or Security levels of Browser, design test cases.

4: Implement & defect Report: Implement the test cases, Checks defects found. If defects founded noted it, so may Useful for future development.

5: Regression testing: Test on the Updated Test Cases, the defect got solved, in order to avoid the defect Causing Again.

6: Test execution: Execute the test with Updated Test Cases, Note down the process (defects).

12) Defect Repositories for Analyzing repeated defects:

Repeated defects occur because of inadequate root cause analysis, incomplete regression testing, poor documentation.

Defect Repositories must have:

→ Defect ID & Title

→ Defect Category & affected module

→ Steps to reproduce Problem

→ Root Cause & Solution

Identifying Defect Patterns:

The team can:

→ Group defects by modules & category

→ Compare defect counts

→ Design additional payment validation

→ Perform Regression testing.

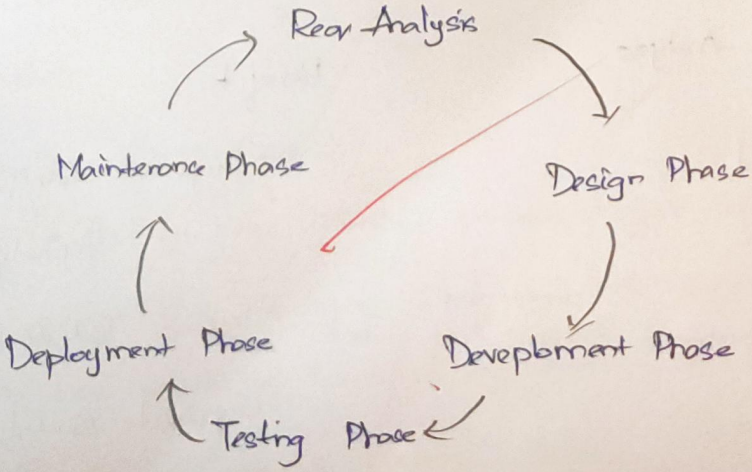
Improvements:

→ Conduct root cause

→ Update Regression

→ Improve Requirements.

SDLC Phase:



Requirement Analysis:

tester Review requirements, Check Requirement is testable.

Design Phase:

Tester review workflow, database, design

Developers phase:

Team collaborates with developers, review test.

Testing phase:

tester executes different test cases.

Deployment Phase:

Deploy performs smoke testing, Crucial functiliee

Maintenance Phase:

test performs regression testing.

---

14) Standarized Testing Process:

1. Requirement Analysis
2. Test Planning
3. Test Design
4. Test Environment Setup
5. Test Execution
6. Defect Management
7. Regression Testing

Benefits:

-> Improves Consistency & Repeatibility

->Increase Test Coverage

-> Support Quality measurement

15)